

The pay run: the screen that finally computes a real paycheque

Prepared for Michael Lyman · Greenway Marijuana · books-39

What changed, in one paragraph

Until this slice, Greenway had every part of a payroll system and no payroll. The timesheet engine could turn punches into hours and gross pay for real employees. The withholding engine could compute federal income tax, Social Security, Medicare, Paid Family and Medical Leave, WA Cares, unemployment and L&I. The garnishment engine could compute every ceiling in the statute. The year-to-date store could tell you where somebody stood against the Social Security wage base. All of it was written, all of it was tested, and **not one line of code joined any of it to an actual person**. I verified that before writing anything: `computeNetPay` was called in exactly one place in the entire repository, and that place was a demonstration on the net-pay illustration screen. The engines were talking to a worked example, not to your staff. This slice builds the join, and it puts it behind a screen at **Books → Pay Run** that tells you, in one sentence at the top of the page, the single next thing to do.

The blocker I found first, because it is the reason this took a whole slice

The withholding engine needs a complete W-4 record for each person before it can compute a single federal income tax figure: the form year, the filing status, whether the multiple-jobs box is ticked, the dependent credit amount, the three step-4 adjustments, any legacy allowances, whether the person claimed exemption, and the date the form was signed. Those columns exist in the database and the onboarding screen writes every one of them correctly.

Nothing ever read them back. Every single read of the `employee_w4` table anywhere in the codebase selected one column — `employee_id` — and used it as an existence check: does this person have a W-4 on file, yes or no. The actual elections went into the database and never came out. There was no function to turn a stored row back into a W-4 record, which meant the withholding engine, which is the largest and most carefully tested module in the payroll domain, had no possible way to be given real data about a real employee. That missing read-back was the whole blocker, and it is now built and tested.

The law this screen is built on, in its own words

I want you to see the actual statutory text rather than my summary of it, because the difference between what a regulation says and what people assume it says is where payroll goes wrong. Five authorities govern the act of running a payroll, and all five are stored in the system with their verbatim text, so the software and this report cannot drift apart.

1. A missing W-4 does not stop payroll — it triggers a specific, named treatment

26 C.F.R. § 31.3402(f)(2)-1(a)(4) says:

"(4) If an employee has no valid withholding allowance certificate in effect with the employer at the time of the payment of the wages, and fails to furnish a valid withholding allowance certificate to the employer, the employee will be treated as single but having the withholding allowance provided in forms, instructions, publications, and other guidance prescribed by the Commissioner."

In plain English: this is the sentence that says a missing W-4 does **not** stop payroll, and it is the reason the pay run has a documented fallback instead of a refusal. If somebody never handed in a W-4, the law does not tell you to withhold nothing and it does not tell you to guess — it tells you to treat them as **single with no adjustments**. That is a specific treatment with a specific dollar consequence, and it is almost always **more** withholding than the employee would have chosen for themselves. The software applies exactly that treatment, labels the cheque as having used it, and tells you to go collect the form — because the employee is the one paying for the missing paperwork out of every cheque until it arrives.

This is the single most valuable sentence in the whole slice, and here is why. Everybody's instinct is that a missing W-4 should block the cheque. It must not. Blocking it would mean not paying somebody for work they have already done, which is its own violation. So the cheque computes, it is arithmetically perfect, it is legally required, and it is quietly costing that employee money every fortnight. Nothing will break to remind you. That is exactly the situation the middle colour on this screen exists for, and I come back to it below.

2. An invalid W-4 is worse than a missing one, and there are three duties hidden in the paragraph

26 C.F.R. § 31.3402(f)(2)-1(e)(1)(ii) says:

"(ii) Employer disregard of invalid withholding allowance certificate. If an employer receives an invalid withholding allowance certificate, the employer must disregard it for purposes of computing withholding. The employer must inform the employee who furnished the certificate that it is invalid and must request another withholding allowance certificate from the employee. If the employee who furnished the invalid certificate fails to comply with the employer's request, the employer must treat the employee as single but having the withholding allowance provided by the forms, instructions, publications, and other guidance prescribed by the Commissioner. If, however, a prior certificate is in effect with respect to the employee, the employer must continue to withhold in accordance with the prior certificate."

Three separate duties hide in that paragraph, and payroll software usually implements none of them. First, an invalid form must be **disregarded** — not partially honoured, not "best-effort" interpreted. Second, you must **tell** the employee it is invalid and ask for another; that is an affirmative obligation on Greenway, not a courtesy. Third — and this is the one everybody misses — if the employee previously filed a **good** form, you keep using the **old** one. You do not fall back to single. The system enforces this by never deleting a superseded W-4: the "is current" flag is flipped and the row stays, so the prior certificate is still there to fall back to. An unsigned form is the common case of "invalid" here, and the software treats a form with no signature date as having no W-4, which is exactly this rule.

3. The employee must furnish it, so chasing it is not nagging

26 C.F.R. § 31.3402(f)(2)-1(a)(1) says:

"(1) On or before the date on which an individual commences employment with an employer, the individual must furnish the employer with a signed withholding allowance certificate (see § 31.3402(f)(5)-1) relating to the filing status the employee reasonably expects to claim under § 31.3402(l)-1(b) for the calendar year for which the withholding allowance certificate is in effect and the withholding allowance under § 31.3402(f)(1)-1(b) that the employee claims."

The word is "must", the deadline is "on or before" the first day, and the duty is the **employee's**. This matters for how the screen talks to you. When the pay run reports a missing W-4 it is not reporting a Greenway failure and it is not asking you to apologise for chasing it — the form was due before that person's first shift. It also means the signature is part of the requirement, not decoration: the text says "a signed withholding allowance certificate". A form with data and no signature has not been furnished.

4. Withheld support money has a five-working-day clock on it

RCW 26.18.110(2) says:

"(2) If the employer possesses any earnings or remuneration due and owing to the obligor, the earnings subject to the wage assignment order or income withholding order shall be withheld immediately upon receipt of the wage assignment order or income withholding order. The withheld earnings shall be delivered to the Washington state support registry or, if the wage assignment order is to satisfy a duty of maintenance, to the addressee specified in the assignment within five working days of each regular pay interval."

The moment a pay run takes support money out of somebody's cheque, a five-working-day clock starts on Greenway to send it on. This is why the pay run does not stop at "here is the net pay" — when an order withheld anything, the result carries the remittance amount, the payee, and the deadline, so the obligation is visible on the same screen that created it. Money withheld and not remitted is the worst position to be in: the employee has already been docked, so they are made whole in nobody's eyes, and Greenway is sitting on funds that belong to a court registry.

5. The minimum wage floor is annual, and it is not only about wages

RCW 49.46.020(2)(b) says:

"(b) On September 30, 2020, and on each following September 30th, the department of labor and industries shall calculate an adjusted minimum wage rate to maintain employee purchasing power by increasing the current year's minimum wage rate by the rate of inflation. The adjusted minimum wage rate shall be calculated to the nearest cent using the consumer price index for urban wage earners and clerical workers, CPI-W, or a successor index, for the twelve months prior to each September 1st as calculated by the United States department of labor. Each adjusted minimum wage rate calculated under this subsection (2)(b) takes effect on the following January 1st."

There are two consequences for a pay run, one obvious and one not. The obvious one: the minimum wage changes every January 1st, so a rate loaded for 2026 is **wrong** for 2027 and the system refuses rather than carrying it forward. The non-obvious one: the minimum wage is an input to **garnishment**, not just to wages. The federal exemption protects thirty times the

minimum hourly wage, and Washington's is higher, so an out-of-date minimum wage silently changes how much can be taken from a garnished employee. That is why a missing minimum wage blocks the whole run and not merely the low-paid cheques.

Your blocker right now, and it is a real one

I ran the readiness check against your actual cutover date. Here is the verbatim answer the system gives for a pay date of **1 January 2027**:

"10 of 11 rates have no evidenced row covering 2027-01-01: WA Paid Leave total premium rate, WA Paid Leave employee share of the premium, WA Paid Leave employer share of the premium, WA unemployment tax rate, WA unemployment wage base, L&I employee hourly rate, L&I employer hourly rate, Social Security wage base, Washington minimum wage, federal minimum wage. Until each one is on file with the notice it came from, no paycheck dated 2027-01-01 can be calculated — the system refuses rather than reusing the prior year's figure."

Ten of your eleven rates are missing for your first payroll. Only the WA Cares premium is on file, and it is on file because that rate is fixed in statute rather than reset annually. **The pay run will refuse outright on 1 January 2027 until you supply the other ten.** That is not a bug and I am not going to soften it: it is the system doing precisely what it was built to do, and it is the difference between a payroll that is wrong for a year and a payroll that will not start until it is right.

What you need to obtain, and roughly when each becomes available: the **2027 L&I rate notice** for risk class 6403 on account 521,756-00, which L&I mails in the autumn and which gives you both the employee and employer hourly rates; the **2027 ESD unemployment tax rate notice** for account 000-073905-00-0, which ESD issues in December and which also carries the new taxable wage base; the **2027 Paid Family and Medical Leave premium rate and the employer/employee split**, which ESD announces around the end of September; the **2027 Social Security wage base**, which the Social Security Administration announces in October; and the **2027 Washington minimum wage**, which under the statute quoted above L&I must calculate by 30 September 2026 for a 1 January 2027 effective date. The federal minimum wage has not moved since 2009 but the system still requires it to be entered with its source, because "it has not changed" is a belief and an evidenced row is a fact.

Every one of those figures is published **before** your cutover. None of them requires you to wait until January. The right time to enter them is the week each notice arrives, and the place is Books → Payroll rates, attaching the notice itself so the number can be traced back to its source years from now.

The seven-check pre-flight list, in the order the screen shows them

The checklist is on the pay run page and it is deliberately ordered. Each item sits where it does for a reason, and the reason is on the screen next to it. Here they are with the reasoning, because the ordering is the part that carries the value.

One: is every rate for this pay date on file, with the notice it came from? This is first because a missing rate is the only problem on the list that is wrong for everybody at once, so it is worth knowing before you look at a single person. It is also the one most likely to be true in January, since L&I, ESD and the PFML premium all reset on 1 January and your first payroll is 1 January 2027. You do not have to check this by hand — the pay run refuses outright and names each missing rate. Note carefully that rates are chosen by **pay date**, not by the period worked, so a period ending in December that pays in January needs the new year's rates. If this fails, nothing is computed at all, which is the point: carrying last year's figure forward produces a small, entirely plausible error on every cheque for a year, and it surfaces as a reconciliation difference twelve months later with no obvious cause.

Two: are the timesheets finished — no open punches, no overlaps? Hours drive gross pay, and gross pay drives everything after it: withholding, the employer's own taxes, and the disposable earnings any garnishment is measured against. A wrong hour figure makes all of them wrong in proportion, so each one still looks internally consistent while all of them are wrong together. Open Timesheets for the period; the pay run reads exactly the figures that screen shows. Anyone the timesheet refused appears as a **blocked** line rather than being quietly left out of the run. If it fails, that person is blocked and nobody else is affected.

Three: does everybody have a signed W-4 — and is it actually signed? A missing W-4 does not stop a cheque, as the regulation above makes clear. The law says exactly what to do without one, so the cheque computes and is arithmetically correct. That is precisely why it needs checking deliberately: nothing will break to remind you. Any line marked **needs attention** says which of the two it is — no form at all, or a form on file that nobody signed. An unsigned W-4 is not a W-4, so the elections written on it are disregarded entirely and the person is withheld as single with no adjustments. If it fails, the employee is over-withheld all year and gets it back as a refund the following April. It is not an employer penalty and it is not an error on your part — but it is somebody's money sitting with the IRS for a year, and they will ask you about it.

Four: is each person's pay frequency right — biweekly for staff, annual for the owner? This one is invisible, and that is why it is on the list. Every other item announces itself; a wrong pay frequency produces a completely ordinary-looking cheque with the wrong income tax on it. Check it at Staffing → the employee → Pay. Greenway's staff are biweekly, which is twenty-six cheques a year; you are annual, which is one. If no frequency is set at all, the line is **blocked** rather than guessed at. The cost of getting it wrong is worth understanding precisely: the federal tables annualise the wages, find the bracket, then divide back down by the number of periods, so believing twenty-four where the truth is twenty-six puts every income-tax figure out by about eight percent, all year, and it will not tie out until the W-2.

Five: has any garnishment or support order been released, changed, or ended? Court orders arrive and end by post, on their own schedule, with no connection to payday. Nothing in the software can know an order was released; only you can. Payroll → Garnishments lists every active order, with suspended and terminated ones excluded from the run entirely — compare it against the paperwork on your desk. The asymmetry here is the point: withholding on a released order is **worse** than missing one, because the money has already gone to somebody not entitled to it, so getting it back means recovering it from them rather than simply correcting a figure. Missing one is recoverable from the next cheque.

Six: for the first run of a year, are the year-to-date figures carried in correctly? Year-to-date decides when the Social Security wage base stops applying. It only matters for people who cross it, and it matters enormously for them. Check Payroll → Year to date. For 1 January 2027 every figure should be zero, because the wage bases reset on the calendar year — that is the one genuine advantage of your 1 January cutover and it is worth a great deal. For a mid-year cutover they would have to match the last payroll run in the old system to the penny. Understated year-to-date keeps withholding Social Security after the ceiling is reached, so the employee over-pays and it must be refunded; overstated stops withholding early, and the employer owes the difference plus the matching share.

Seven: have you actually read three finished cheques, line by line? This is last because it is the only check that can catch a category of error nobody predicted — the six above test things somebody thought of in advance. Pick the highest-paid person, the lowest-paid person, and anyone with a garnishment. Read every line and ask whether it is roughly the size you expected. You are not re-doing the arithmetic; you are checking that nothing is an order of magnitude out. If a figure looks wrong, it probably is: stop the run rather than paying it. Nothing has been saved until you approve, so stopping costs nothing but a delay.

Four of those seven cannot be answered by software, and the screen does not pretend otherwise. Nothing in the database knows whether an order was released last Tuesday or whether you read three cheques end to end. Those four render as "you must confirm" and are never auto-ticked. I want to be explicit about why: a checklist that ticks itself is a checklist nobody reads, and its ticks are worth nothing to an examiner.

The seven ways the pay run will refuse, and what each one means

The system will refuse to produce a figure rather than produce a wrong one. There are exactly seven reasons it will do that, each with its own plain-English lesson on the screen.

NO_HOURS — the timesheet could not produce hours for this person. Hours are the first number in the chain; everything after them is computed from them, so there is no honest figure to print without them. The usual cause is an open punch: somebody clocked in and never clocked out. Treating a missing punch as zero hours pays somebody nothing for a day they worked, and it looks exactly like a short week rather than like an error.

NO_GROSS — there are hours, but no pay rate to value them at. Hours without a rate is not zero pay, it is unknown pay, and the difference between unknown and zero is the whole reason this stops. Set the basis and the rate at Staffing → the employee → Pay. A zero rate would produce a cheque for \$0.00 that is internally consistent — taxes of zero on gross of zero all reconcile perfectly — and would be paid without anything looking wrong until the employee asked where their wages went.

NO_PAY_FREQUENCY — no current pay record says how often this person is paid. This is a separate code from NO_GROSS on purpose, because it is a divisor rather than a missing amount. Assuming biweekly for you, when you are paid annually, would annualise your salary twenty-six times over, land it in the top bracket, and withhold an enormous figure that looks entirely plausible on screen because nothing about it is arithmetically inconsistent.

RATE_NOT_ON_FILE — a rate every cheque needs has no evidenced row for this pay date. This is the one blocking you today, ten times over. These premiums come out of the employee's own cheque, so computing without one withholds nothing for it, which overstates take-home pay **and** overstates the disposable earnings any garnishment is measured against. This is not hypothetical: carrying a missing PFML rate through as zero made a test cheque \$19.37 too high, and because a creditor garnishment takes twenty-five percent of disposable earnings, it over-garnished somebody who was already being garnished. It survived review and was caught only by running it.

NO_MINIMUM_WAGE — the minimum wage in force on this pay date is not on file. The protected floor, the amount a garnishment may never take a person below, is a multiple of the minimum wage. No minimum wage means no floor, and no

floor means every garnishment computed would take too much from the lowest-paid people on the payroll. Reusing last year's Washington figure under-protects every garnished employee by the amount of the annual increase, quietly, on every cheque until somebody notices.

ENGINE_REFUSED — the net-pay engine declined to produce a figure, and said why. Its own reasons are carried through word for word rather than summarised, because re-wording them would give one problem two different descriptions in two places. Most often it is one of the two garnishment questions: whether the employee supports another family, and whether any arrears are more than twelve weeks old. Those two answers are the difference between a fifty percent and a sixty-five percent ceiling on somebody's wages, and defaulting either one silently under-withholds child support — the one error in this whole domain that lands on the employer personally.

DOES_NOT_RECONCILE — the parts of this cheque do not add back to the whole. Gross, less every deduction, must equal net exactly. If it does not, one of the figures is wrong and there is no way to tell which from the outside. **This should never appear.** If it does, do not pay the run: it indicates a defect in the software rather than in your data, so nothing you change on a screen will fix it. Record which employee and which pay date produced it — that is what makes it findable. A cheque that does not reconcile will not tie to the 941, the W-2, or the bank, and the error compounds through every filing that reads it.

Three colours, and why the middle one exists

The screen shows every person in one of three states, and the fact that there are three rather than two is the most important design decision in this slice.

Green means ready: "This cheque is computed and nothing is outstanding."

Gold means needs attention: "Pay this cheque — it is correct and it is legally required. There is also something to fix, listed below, and fixing it does not change this cheque."

Red means blocked: "No honest figure can be produced for this person, so none is shown. Nothing is guessed and nothing is part-paid."

The gold state is the whole point. Go back to the first regulation: an employee with no W-4 produces a perfectly valid, legally required cheque. You cannot block it — refusing to pay somebody for work they have done is its own violation. But if you call it *ready*, that person is over-withheld for eleven months and nobody ever looks at it again, because green means finished and green means scroll past. Gold means pay it and also do something about it. That argument is worth nothing if the stylesheet renders gold and green as the same colour, so there is a test asserting the three states map to three genuinely different tones.

Attention is gold rather than orange for a specific reason: orange reads as "this number is wrong", and the number is not wrong. The paperwork is.

Two more presentation rules are enforced by tests rather than merely intended. **Money is shown as a dash when a line is blocked, never as \$0.00**, because a zero looks like a real cheque for nothing — the same reasoning the rate reader uses when it refuses to substitute zero for a missing rate. And **the net-pay total warns you when it is partial:** the total sums only the payable lines, so on a run with any blockage it is not what will leave the bank, and the screen says "This is NOT the full cost of the period... Do not plan your cash against this figure." That sentence is the difference between a number and a cash-planning error.

At the very top of the page, above everything, is **one sentence and one destination** — the single next action, chosen by a strict priority order that puts a store failure first, then a locked period, then blocked people (naming every one of them), then an unreadable W-4, then an empty run, then attention items, then the all-clear. A payroll screen showing eleven equally-weighted problems is a screen that gets scrolled past.

If you have already run it wrong: the five-stage recovery ladder

This is on the screen too, and it is shown even when the page is happy, because the moment you need it is the moment something has already gone wrong and you will not be in the mood to go looking for it. Find the stage you are actually at, and note the trap attached to each one — the traps are the part people get wrong.

Stage one — you are looking at the run and have not approved it. Nothing has been saved. Just fix it and look again: correct whatever is wrong at its source (the punch, the rate, the W-4, the order) and re-open the pay run. It is recomputed from scratch every time you open it, so there is no stale copy to clear and nothing to undo. *The trap:* trying to correct a figure **on** the pay run screen instead of at its source. There is deliberately no way to type over a computed number here. A cheque you can

hand-edit is a cheque nobody can reproduce later, which is exactly the position the old spreadsheet left you in.

Stage two — the run is approved but no money has left the bank. Still recoverable cleanly, but reverse before you re-run. Reverse the run so the year-to-date accumulators go back to where they were, then fix the source and run it again. Do **not** simply run a second time on top — that adds a second set of wages to the year-to-date and the wage bases will be wrong for the rest of the year. *The trap:* assuming that because no money moved, nothing was recorded. The year-to-date figures were updated at approval, and they are what every ceiling for the rest of the year is measured against.

Stage three — employees have been paid, and the quarter is not filed yet. Fix it on the next cheque, in the same quarter, and keep the paper. If you under-withheld, recover it from the next cheque within the same quarter so the 941 for the quarter is right as filed. If you over-withheld, refund it on the next cheque. Either way write down what happened and why, and tell the employee before their next payslip looks strange to them. *The trap:* crossing a quarter boundary while catching up. A correction made in April for a March error means the Q1 941 was filed wrong and the Q2 941 is now also wrong. Inside one quarter it is a self-correcting adjustment; across two it is an amended return.

Stage four — the 941 for that quarter has already gone in. This is a 941-X. It is routine, and it is not optional. Form 941-X has its own deadlines and its own rules about whether you are correcting an underpayment or claiming a refund, and those two paths differ. This is the point at which the question is worth an hour of a CPA's time rather than an afternoon of yours. *The trap:* quietly absorbing a small under-withholding rather than filing. The amount is not the issue — the mismatch between what was deposited and what the wage records show is the issue, and it is precisely what a payroll examination compares.

Stage five — W-2s are out and the employee has probably filed. You need a W-2c, and the employee needs to know immediately. Issue a corrected W-2c and send it to the employee and to the Social Security Administration. If they have already filed a return using the original, they may need to amend it, and that is a cost your error has imposed on them. *The trap:* waiting until next January to fix it on the next W-2. Wages belong to the year they were **paid**. Moving them to a later year to tidy up the paperwork misstates two years instead of one.

What the approve button does today, and why it says so

The approve button on the pay run screen is rendered **disabled**, and it says "Not connected yet" rather than pretending. Approving a pay run is a write with its own audit trail, its own permission check and its own confirmation step, and it lands in the next slice. A button that half-works is worse than one that honestly states what it is waiting for, particularly on a screen that moves money.

Everything upstream of the button is real. The run computes, refuses, explains, and reconciles against live data.

Four defects this slice found and fixed on the way through

I want these on the record, because each one was invisible and each one was found by building something that had to use the code rather than merely test it.

Three dangling citations. Every W-4 lesson cited its legal authority by the TypeScript variable name rather than the actual authority identifier. The compiler was silent because both are strings. On screen each one read as though a lawyer had checked the sentence beneath it, and none of them resolved to anything at all. The instance fix was three corrected strings; the class fix was making the identifier a closed set so the wrong value no longer compiles.

A parser that stopped reading at a semicolon inside a comment. The tool that checks every refusal code has a lesson attached was truncating its scan at the first semicolon, which happened to fall inside a documentation comment. The consequence was that `DOES_NOT_RECONCILE` — the code that means a cheque does not add up — was invisible to the checker. Fixed in the shared parser, not locally.

An authority registry that was never merged into the guidance system. The five statutes above were mirrored, verbatim-checked, and registered nowhere, so no screen could have surfaced them.

A tripwire that could not tell code from prose. The check that catches citations pointing at nothing was reading legal text out of documentation comments. I measured before changing anything: 1,220 citations in raw source, 1,219 after comments are stripped, exactly one living in prose, and zero unresolved in real code. I fixed it by making the scan comment-blind rather than by rewording the comment, because rewording it would have created an incentive to stop documenting mistakes.

There was also a bug in the store that would have cost real money: the pay frequency fell back to "biweekly" when none was on file, underneath a comment that explicitly said it must never do that. That is fixed at the type level and is now the `NO_PAY_FREQUENCY` refusal described above.

How this was verified

Sixty-one new tests cover the screen logic, on top of the tests for the join, the read-back, the store and the mentor. This report itself has forty-nine tests of its own, which re-derive every statutory quotation above from the stored text and recompute every number in it.

Beyond counting tests, I ran a sixteen-mutation battery against the screen: I deliberately broke the code sixteen ways — rendering attention as green, auto-ticking the check that cannot be auto-ticked, making a locked period approvable, making a blocked run approvable, showing \$0.00 instead of a dash, deleting the partial-total warning, dropping refusals from the cards, demoting the unreadable-W-4 case, hiding a zero count, deleting the menu entry, removing the access check, having the page bypass its own logic, leaving blocked people unnamed, giving two failure codes the same headline, letting the pure logic read the clock, and removing the words "pay this cheque" from the front of the attention sentence. **All sixteen were caught by the test suite.** One of them initially reported as surviving, and it turned out my mutation script had used a hyphen where the source has an em-dash, so the mutation never applied at all.

That happened a second time, on this report, in a new disguise, and I am recording it because the lesson generalises. I ran a seventeen-mutation battery against the checks on this document, and it reported all seventeen caught. It was lying. The restore step used a command that only works on files already known to source control, and this report was brand new, so nothing was ever restored — the mutations piled on top of each other and only the first result meant anything. I rebuilt the battery to snapshot every file it touches to a scratch directory first, verify after every single restore that the file came back byte-for-byte, and abort the whole run the instant one does not. Re-run that way, all seventeen are genuinely caught.

The general lesson is worth more than either incident: **a check that silently does nothing is indistinguishable from a check that passes.** That is true of a mutation that never applied, of a restore that never restored, and of a test that scans an empty string. Every one of them shows you green. It is the reason so many of the tests in this project spend their first few assertions proving that the test itself is capable of failing before they test anything real.

One test was deliberately **narrowed** after I proved it was overreaching. It originally asserted that every menu icon in the Accounting section is distinct, which fails on the main branch without any of my work — there are two pre-existing duplicate icons. Those are not mine to fix in this pull request, so the test now asserts that my icon is unused and adds a ratchet pinning the duplicate count at two, so no new collision can be introduced without the suite going red.

Finally, and this is the reason the menu entry is part of this work rather than a later tidy-up: phases three through six of this slice produced a store, a join, a mentor layer and over a hundred passing tests, all green, and **none of it was reachable by a human being** because no menu entry pointed at it. Passing tests on a screen nobody can open is the most convincing kind of nothing.

What I need from you, in priority order

The ten missing 2027 rates are the only thing standing between you and a computable 1 January 2027 payroll, and every one of them is published before your cutover. In rough order of when they become available: the **2027 Washington minimum wage** (L&I must calculate it by 30 September 2026), the **2027 PFML premium rate and its employer/employee split** (ESD, around the end of September), the **2027 Social Security wage base** (SSA, October), the **2027 L&I rate notice** for risk class 6403 (autumn, giving both hourly rates), and the **2027 ESD unemployment tax rate notice** for your account (December, giving both the rate and the wage base). The federal minimum wage needs entering with its source even though it has not moved.

Send me each notice as it arrives rather than saving them up. Each one you give me removes one refusal, and the last one turns the pay run on.

Prepared by your development team. Every statute quoted above is stored verbatim in the software with its source, and a test in the build re-derives each quotation in this document from that stored text — so if the two ever disagree, the build fails rather than this report quietly going stale.